

TP Final - Jour 5 : Analyse Complète d'un Ransomware

Type : Projet d'analyse et reverse engineering

Contexte

Vous êtes une équipe d'analystes en cybersécurité. Une entreprise a été victime d'une attaque ransomware et fait appel à vous pour :

1. 🔍 Analyser le malware qui a chiffré leurs données
2. 🗝️ Récupérer les fichiers chiffrés
3. 🌐 Identifier et analyser le serveur Command & Control (C2)
4. 🛡️ Proposer des recommandations de sécurité

Vous disposez d'**une journée complète** pour mener cette analyse.

Objectifs Pédagogiques

À la fin de ce TP, vous serez capables de :

- ✅ Analyser un binaire C (strings, code source, IOCs)
- ✅ Utiliser GDB pour extraire une clé de chiffrement en mémoire
- ✅ Comprendre les communications d'un ransomware avec son C2
- ✅ Créer un outil de déchiffrement (Python ou C)
- ✅ Identifier des vulnérabilités web basiques (SQLi, IDOR)
- ✅ Rédiger un rapport technique synthétique

Niveau : Débutant/Intermédiaire - Code C simple et lisible fourni

Structure du Projet

Le projet est divisé en **5 étapes progressives** :

Étape 1 : Analyse Statique	→ IOCs et reconnaissance
Étape 2 : Debug Dynamique	→ Extraction de clés
Étape 3 : Analyse du Serveur C2	→ Communications réseau
Étape 4 : Recouvrement de Données	→ Déchiffrement
Étape 5 : Pentest Web	→ Vulnérabilités du C2

Chaque étape est **indépendante mais complémentaire**. Vous pouvez les réaliser dans l'ordre ou en parallèle selon votre organisation.

Démarrage Rapide

Prérequis Techniques

Outils nécessaires :

```
# Outils d'analyse
sudo apt-get update
sudo apt-get install -y gdb strace tcpdump wireshark curl python3 python3-venv python3-full

# Note: Les bibliothèques Python seront installées dans un environnement virtuel
# Voir ENVIRONNEMENT_PYTHON.md pour les détails
```

⚠ **Important - Environnement Python** : Sur les distributions Linux modernes, `pip3 install` système n'est plus autorisé. Vous devez utiliser un **environnement virtuel** (venv) pour chaque étape Python.

👉 **Voir le guide complet** : [ENVIRONNEMENT_PYTHON.md](#)

Connaissances requises :

- Bases de Linux (commandes shell : `ls`, `cat`, `grep`)
- Lecture de code C **basique** (if, for, fonctions, pointeurs simples)
- Notions de réseau (HTTP, ce qu'est une requête)
- GDB basique (vu en Jour 3-4 : breakpoints, print, continue)

Étape 1 : Analyse Statique 🔍

Objectif : Effectuer une reconnaissance initiale du ransomware (binaire C)

Ce que vous allez faire :

- Calculer les hashes (MD5, SHA256) de l'échantillon
- Extraire les strings avec `strings` et `grep`
- Analyser le code source C fourni (pour comprendre la logique)
- Trouver les adresses IP, domaines, ports
- Identifier l'algorithme de chiffrement utilisé

Commandes à utiliser :

```
# Analyser le binaire compilé
md5sum ransomware
sha256sum ransomware
file ransomware
strings ransomware | grep -i "http"
strings ransomware | grep -i "\.enc"

# Lire le code source C (fourni pour faciliter l'analyse)
```

```
cat ransomware.c | grep -E "encrypt|key|http"
grep -n "main" ransomware.c # Trouver la fonction principale
```

Livrables attendus :

- **IOCs.txt** : Hashes, IPs, domaines, ports
- Liste des fonctions importantes (encrypt_file, send_key, etc.)
- Type de chiffrement identifié (XOR, AES, RC4...)

📁 **Dossier** : `Étape1_AnalyseStatique/`

Étape 2 : Debug & Extraction 🐛

Objectif : Extraire la clé de chiffrement avec GDB

Ce que vous allez faire :

- Lancer le ransomware dans GDB
- Placer un breakpoint sur la fonction de chiffrement
- Inspecter la mémoire pour trouver la clé
- Extraire la clé en clair

Étapes guidées :

```
# 1. Lancer GDB
gdb ./ransomware

# 2. Placer un breakpoint sur la fonction encrypt_file
(gdb) break encrypt_file

# 3. Lancer le programme
(gdb) run

# 4. Afficher la clé (variable 'key' ou 'encryption_key')
(gdb) print key
(gdb) x/s key          # Afficher comme string
(gdb) x/32xb key       # Afficher 32 bytes en hexa

# 5. Continuer l'exécution
(gdb) continue
```

Aide : Si vous voyez dans le code source `char key[] = "..."`, vous pouvez trouver cette variable en mémoire !

Livrables attendus :

- La **clé de chiffrement** complète (noter tous les caractères)
- Commandes GDB utilisées
- Capture d'écran de GDB avec la clé visible

📁 Dossier : Étape2_DebugDynamique/

Étape 3 : Analyse du Serveur C2 🌐

Objectif : Comprendre comment le ransomware communique avec son serveur

Ce que vous allez faire :

- Lire les logs réseau déjà capturés (fournis)
- OU capturer le trafic avec `tcpdump` en lançant le ransomware
- Identifier les endpoints de l'API dans le code C
- Comprendre ce qui est envoyé (ID victime, clé, hostname, etc.)

Commandes utiles :

```
# Chercher les URL dans le code C
grep -n "http://" ransomware.c
grep -n "POST\|GET" ransomware.c

# Chercher les fonctions réseau
grep -n "curl\|socket\|send" ransomware.c

# Lancer une capture pendant l'exécution (optionnel)
sudo tcpdump -i lo -w capture.pcap port 5000 &
./ransomware # Lancer dans un dossier de test
sudo pkill tcpdump
strings capture.pcap | grep "POST\|GET"
```

Livrables attendus :

- **Schéma simple** : Ransomware → C2 (quelles infos sont envoyées ?)
- Liste de 3-5 endpoints avec leur rôle (ex: `/register`, `/key`)
- Exemple d'une requête HTTP capturée

📁 Dossier : Étape3_AnalyseC2/

Étape 4 : Recouvrement de Données 🔒

Objectif : Créer un outil pour déchiffrer les fichiers (Python ou C au choix)

Ce que vous allez faire :

- Utiliser la clé trouvée à l'Étape 2
- Comprendre l'algorithme utilisé (en lisant le code C du ransomware)
- Compléter le template fourni (Python recommandé, 80% fait)
- Tester sur les 3 fichiers chiffrés fournis

Option 1 - Python (recommandé) :

Template fourni avec 80% du code déjà écrit :

```
# recovery_tool.py
# TODO: Mettre la clé trouvée à l'Étape 2
key = b"VOTRE_CLE_ICI"

# TODO: Lire le fichier chiffré
# TODO: Appliquer l'algorithme inverse
# TODO: Sauvegarder le fichier déchiffré
```

Option 2 - C (pour les plus à l'aise) :

Vous pouvez adapter le code du ransomware en inversant la logique.

Livrables attendus :

- Script `recovery_tool.py` (ou `.c`) complété et commenté
- Les 3 fichiers déchiffrés : `document.txt`, `image.jpg`, `data.pdf`
- Preuve qu'ils s'ouvrent correctement
- Explication : "J'ai utilisé [algorithme] avec la clé [...]"

📁 Dossier : `Étape4_Recouvrement/`

Étape 5 : Pentest Web du Serveur C2 🗝️

Objectif : Trouver des failles de sécurité dans l'application web du C2

Ce que vous allez faire :

- Le serveur C2 Flask est déjà lancé (port 5000)
- Tester les endpoints avec `curl` ou le navigateur
- Chercher : SQLi, IDOR, pas d'authentification, secrets hardcodés
- Extraire des données (liste des victimes, clés stockées)

Exemples de tests :

```
# Test IDOR : changer l'ID
curl http://localhost:5000/api/victim/1
curl http://localhost:5000/api/victim/2

# Test SQL Injection
curl "http://localhost:5000/api/search?name=admin' OR '1'='1"

# Test pas d'auth
curl http://localhost:5000/api/admin/keys
```

Livrables attendus :

- **3 vulnérabilités minimum** avec description
- 2 POCs fonctionnels (commandes curl qui marchent)
- Données sensibles extraites (liste victimes, clés, etc.)

📁 **Dossier** : [Étape5_PentestWeb/](#)

Rapport Final

Structure Simplifiée pour 1 Journée

Votre rapport doit contenir **au minimum** :

1. Page de garde

- Titre du projet
- Noms des membres de l'équipe
- Date

2. Résumé Exécutif

- Nature du ransomware
- Résultats clés (clé trouvée, vulnérabilités, etc.)

3. Analyse Technique

- **Étape 1** : IOCs principaux (hashes, IPs, fonctions)
- **Étape 2** : Méthode d'extraction de la clé
- **Étape 3** : Schéma simple du C2 + endpoints
- **Étape 4** : Algorithme de déchiffrement utilisé
- **Étape 5** : 3 vulnérabilités trouvées avec impact

4. Recommandations

- 3-5 recommandations de sécurité concrètes
- Outils de détection suggérés

5. Annexes

- Code source du recovery tool
- Liste complète des IOCs
- 2-3 captures d'écran clés

Format du Rapport

- **Format** : PDF ou Markdown (selon le temps)
- **Qualité** : Clair et structuré (pas besoin de mise en page parfaite)
- **Contenu** : Privilégiez la technique à la forme

Template fourni : Voir [Rapport_Final/template_rapport.md](#)

⚠ **Conseil** : Prenez des notes tout au long de la journée pour gagner du temps !

Livrables à Remettre

 **DATE LIMITE : Aujourd'hui 17h00** à Remettre

1.  `rapport_final.pdf` ou `.md`
2.  `recovery_tool.py` (code source commenté)
3.  `IOCs.txt` (liste des indicateurs de compromission)
4.  Notes sur les vulnérabilités web trouvées

Format de remise :

- Déposer dans le dossier partagé ou envoyer par email
-

Bonus (Si vous avez le temps)

Si vous finissez en avance, vous pouvez :

Trouver plus de vulnérabilités web

- 3 vulnérabilités = bien, 5+ = excellent

Améliorer votre recovery tool

- Gestion d'erreurs robuste
- Support de plusieurs algorithmes
- Interface utilisateur

Analyse MITRE ATT&CK

- Mappez 5-10 techniques du ransomware au framework MITRE

Règles de détection simples

- Créez 2-3 règles YARA basiques pour détecter ce ransomware
-

Règles Importantes

Travail en Équipe

- **Équipes de 3-4 personnes**
- Chaque membre doit participer activement

Intégrité Académique

Interdit :

- Copier le travail d'une autre équipe
- Télécharger des solutions toutes faites

Autorisé :

- Consulter la documentation officielle

- Utiliser Stack Overflow pour des questions précises
- Demander de l'aide à l'enseignant
- S'inspirer de tutoriels (avec citation)

Environnement de Test

⚠ **ATTENTION** : Vous travaillez sur des malwares fonctionnels !

Règles de sécurité :

- ☒ Utilisez une VM isolée (sans snapshots importants)
- ☒ Déconnectez le réseau si pas nécessaire
- ☒ Ne testez JAMAIS sur votre machine personnelle
- ☒ Ne diffusez pas les échantillons en dehors du cours

En cas de problème : Contactez immédiatement l'enseignant

☒ Checklist Finale

Avant de rendre votre projet, vérifiez :

Livrables

- ☐ Rapport au format PDF
- ☐ Code source du recovery tool (commenté)
- ☐ Fichier IOCs.txt complet
- ☐ Rapport de pentest web
- ☐ Schéma d'architecture

Rapport

- ☐ Page de garde avec noms
- ☐ Sommaire avec numérotation
- ☐ Pas de fautes d'orthographe majeures
- ☐ Schémas et captures légendés
- ☐ Annexes complètes

Code

- ☐ Le code s'exécute sans erreur
- ☐ Commentaires en français
- ☐ README ou guide d'utilisation
- ☐ Gestion des erreurs basiques